

CESE5040

High-Performance and AI Architectures

2026 Q4

Student Information

Name: Daniel Tyukov

Student Number: 5714699

Lab: Lab Assignment 4

Notes

- Hardware: local laptop, Intel Core Ultra 9 185H, 16 cores / 22 threads, 62 GiB RAM. No NVIDIA GPU on this machine, so every JAX run is on the CPU backend. The MLP and LeNet-5 models are small enough that this is fine for the assignment.
- Software: Python 3.12, JAX 0.10.0 (CPU), Flax 0.12.7, Optax 0.2.8, TensorFlow-CPU for the MNIST loader.
- Submitted scripts: `L4_Q4.py` (drives every Q4.4 sub-experiment) and `L4_Q6.py` (LeNet-5 implementation plus its sweep). The Q4.5 numbers were produced by a separate runner that wraps the supplied `mlp_adv.py` architecture across three seeds; the architecture itself is unchanged.
- Each accuracy figure is a single run at the stated seed. Wall times are wall-clock seconds reported by `time.perf_counter()` around the training loop only.

Lab Exercise 4

Question 4.1 - Backprop by hand and automatic differentiation

1. Let $z_1^1 = w_{11}^1 x_1 + w_{21}^1 x_2 + b_1^1$, $z_2^1 = w_{12}^1 x_1 + w_{22}^1 x_2 + b_2^1$, and $z^2 = w_{11}^2 h_1 + w_{21}^2 h_2 + b_1^2$, with $h_i = \text{ReLU}(z_i^1)$ and $y = \sigma(z^2)$. The loss is $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$, so $\partial \mathcal{L} / \partial y = (y - \hat{y})$ and $\partial y / \partial z^2 = y(1 - y)$ from the sigmoid derivative.

For w_{11}^2 the chain runs $\mathcal{L} \rightarrow y \rightarrow z^2 \rightarrow w_{11}^2$:

$$\frac{\partial \mathcal{L}}{\partial w_{11}^2} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial z^2} \frac{\partial z^2}{\partial w_{11}^2} = (y - \hat{y}) y(1 - y) h_1.$$

For w_{21}^1 the chain goes one layer further back, through h_1 :

$$\frac{\partial \mathcal{L}}{\partial w_{21}^1} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial z^2} \frac{\partial z^2}{\partial h_1} \frac{\partial h_1}{\partial z_1^1} \frac{\partial z_1^1}{\partial w_{21}^1} = (y - \hat{y}) y(1 - y) w_{11}^2 \mathbb{I}[z_1^1 > 0] x_2.$$

The ReLU derivative is the indicator $\mathbb{I}[z_1^1 > 0]$, which is why dead ReLU units block any gradient signal from reaching w_{21}^1 .

2. Automatic differentiation (AD) frameworks compute derivatives by walking a computational graph that records every elementary operation (and its known local Jacobian) performed during the forward pass. Two modes exist: forward mode computes one column of the Jacobian per pass and is efficient when the output dimension exceeds the input dimension, and reverse mode (backpropagation) computes one row per pass and is the right choice for scalar-loss training where the input dimension (parameter count) is huge. JAX, PyTorch and TensorFlow all implement reverse-mode AD by tracing the user's Python function into an intermediate representation, then applying the chain rule along the recorded edges in topological order. The benefit is that the user only writes the forward computation; the framework derives gradients exactly (no finite differencing) and at near-forward cost, which makes complex architectures tractable without any hand derivation.

Question 4.2 - MLP parameter count and CNN motivation

1. With I input neurons, L hidden layers of width H , and O outputs, the layer-by-layer count is:

- Input to first hidden: $IH + H$ (weights plus bias).
- Between consecutive hidden layers: $H^2 + H$, repeated $L-1$ times.
- Last hidden to output: $HO + O$.

$$\text{\#params} = IH + H + (L-1)(H^2 + H) + HO + O.$$

For an MNIST-sized example with $I = 784$, $L = 2$, $H = 512$, $O = 10$ this gives about 670k parameters, which is already large for a simple toy.

2. Scaling fully-connected layers up is not feasible for harder vision tasks. A single $224 \times 224 \times 3$ ImageNet image flattened into a $H = 4096$ hidden layer needs $\approx 617\text{M}$ parameters in just one weight matrix, so memory and compute explode roughly quadratically with input size. CNNs sidestep this by exploiting two structural priors. First, locality: a feature like an edge is defined by a small neighbourhood of pixels, so each filter only looks at a $k \times k$ window. Second, translation invariance: the same edge detector should fire wherever the edge appears, so the filter is applied as a sliding window with shared weights. Together this shrinks the parameter count from $H_{\text{in}} \cdot H_{\text{out}}$ down to $k \cdot k \cdot C_{\text{in}} \cdot C_{\text{out}}$, which is independent of input resolution and several orders of magnitude smaller for typical $k = 3$ or 5 .

Question 4.3 - Batching, learning rate, fit quality, loss vs accuracy

1. Two reasons dominate. Averaging the loss over a batch reduces the variance of the gradient estimate, so each step points in a more reliable direction than a single noisy sample would. The second reason is hardware: GPUs and modern CPUs need wide tensors to fill their SIMD or tensor units, so a batch of 128 images runs almost as fast as a single image but does 128 times the useful work. A subtler effect is that very large batches lose the regularising noise of stochastic updates and tend to settle into sharper minima

that generalise worse. How the dataset is split into batches also matters. A fixed deterministic order lets the model exploit the order, so the practice is to reshuffle every epoch. If classes are imbalanced, a batch dominated by one class biases the gradient toward that class.

2. If the learning rate α is too large the parameter updates overshoot the loss minimum at every step and the loss oscillates or diverges, often visible as NaN gradients within a few steps. If α is too small the model still moves in the right direction but the loss drops so slowly that it never reaches a useful minimum within a feasible number of epochs. A scheduled learning rate combines the best of both: start large to make quick progress out of the random initialisation, then drop the rate (step decay, exponential decay, cosine schedule, or one-cycle) so the optimiser can settle into a flat region without bouncing around. Warmup at the very start also helps when the gradients are still poorly conditioned.

3. Overfitting means the network has memorised the training set, including its noise, so training accuracy is high while test accuracy stalls or drops. The signature is a widening gap between training and validation curves: training loss keeps falling and validation loss rises again. Underfitting is the opposite, both training and test accuracy stay low because the model is not expressive enough or has not trained long enough; the loss curve plateaus high. Common remedies are regularisation (L2, dropout), early stopping, or more data on the overfit side, and a larger model or more epochs on the underfit side.

4. The loss is a continuous, differentiable proxy for whatever discrete metric (here, top-1 accuracy) we actually care about. It measures the probability the model placed on the correct class, so it is sensitive to the margin of every prediction, while accuracy is binary per example. A model can keep pushing the cross-entropy down by becoming overconfident on examples it already classifies correctly. That extra confidence does nothing for accuracy and in fact tends to coincide with overfitting, where training loss keeps dropping while test accuracy plateaus. That is why the early-stopping signal of choice is validation accuracy or validation loss, not training loss.

Question 4.4 - Simple MLP on MNIST (L4_Q4.py)

1. *Trained vs untrained accuracy at three seeds.*

Seed	Untrained acc (%)	Train acc (%)	Test acc (%)
0	10.06	97.71	97.08
10	9.41	97.74	97.10
42	5.84	97.87	97.25

Table 1: Per-sample SGD, $\alpha = 10^{-3}$, 10 epochs. Test accuracy spans only 0.17 pp across seeds.

The untrained model sits at chance level (about 10% for ten classes) for every seed, with seed 42 happening to land slightly worse on the test set by random init. After ten epochs all three seeds reach the same neighbourhood, a fraction of a percent apart on test accuracy. The seed shifts the initial weights and the trajectory taken through parameter space but not the final operating point, so trained accuracy is essentially seed-independent for this simple model.

2. *Effect of removing the [0,1] normalization.*

Disabling the `x / 255.0` step at seed 10 collapsed test accuracy from 97.10% to **11.34%** (train 11.24%),

barely above chance. With pixel values left in $[0, 255]$ the first layer sees inputs roughly two orders of magnitude larger than what the default Flax LeCun-normal initialisation was designed for. Pre-activations are then huge, the cross-entropy gradient saturates, and the per-sample SGD step blows past any usable region of parameter space in the first epoch. Normalising to $[0, 1]$ keeps activation statistics inside the regime where ReLU and the default initialisation cooperate.

3. Learning-rate sweep at $\alpha \in \{10^{-1}, 10^{-4}, 10^{-7}\}$, seed 10.

α	Train acc (%)	Test acc (%)
10^{-1}	81.60	82.13
10^{-3} (baseline)	97.74	97.10
10^{-4}	92.14	92.55
10^{-7}	11.39	11.17

Table 2: Per-sample SGD, 10 epochs. The 10^{-3} row is reused from 4.4.1 for context.

At $\alpha = 0.1$ each update is so aggressive that single-example gradients overshoot the loss minimum repeatedly; the model still learns the easy digits but never settles, ending 15 pp below baseline. At $\alpha = 10^{-4}$ the optimiser is on a healthy trajectory but ten epochs are not enough to reach the baseline. At $\alpha = 10^{-7}$ the steps are so small that the parameters barely move and the model stays at random-init accuracy.

4. Mini-batching for batch sizes 32, 64, 128, 256, 512.

Batch	Train acc (%)	Test acc (%)	Wall (s)
32	89.39	89.93	4.5
64	87.02	87.75	3.6
128	83.34	84.28	2.4
256	77.22	78.18	1.8
512	67.84	68.43	1.4

Table 3: $\alpha = 10^{-3}$, 10 epochs, seed 10. Compare against the per-sample baseline at the same α (97.10% test, 85 s wall).

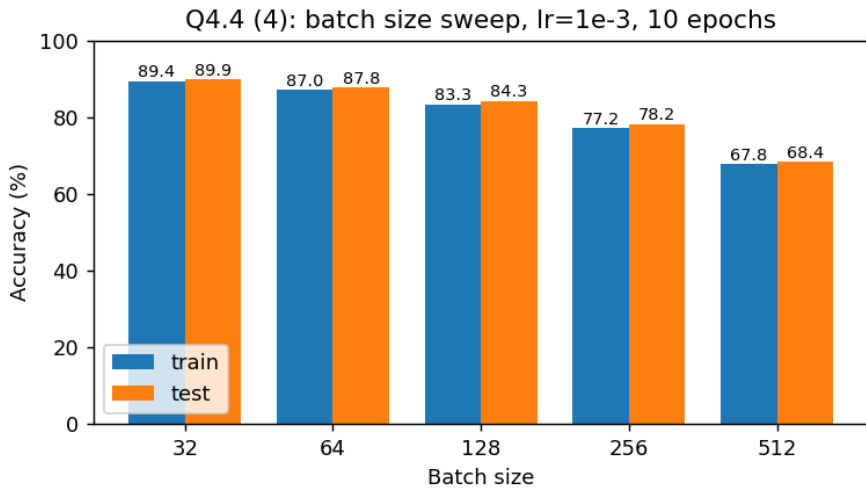


Figure 1: Test accuracy falls monotonically as batch size grows, at fixed α .

Two things happen at once. Wall time drops because each batch is one JAX dispatch carrying more work, and accuracy also drops because the effective number of weight updates per epoch shrinks from 60,000 (per-sample) to 117 (batch 512). With α held at 10^{-3} those few updates are simply not enough; this is the textbook setup for the linear scaling rule of learning rate to batch size, which 4.4.5 verifies.

5. Increasing the learning rate for batch size 64.

α	Train acc (%)	Test acc (%)
10^{-3} (baseline)	87.02	87.75
10^{-2}	93.37	93.40
10^{-1}	98.40	97.51
0.5	99.35	97.65

Table 4: Batch 64, seed 10, 10 epochs.

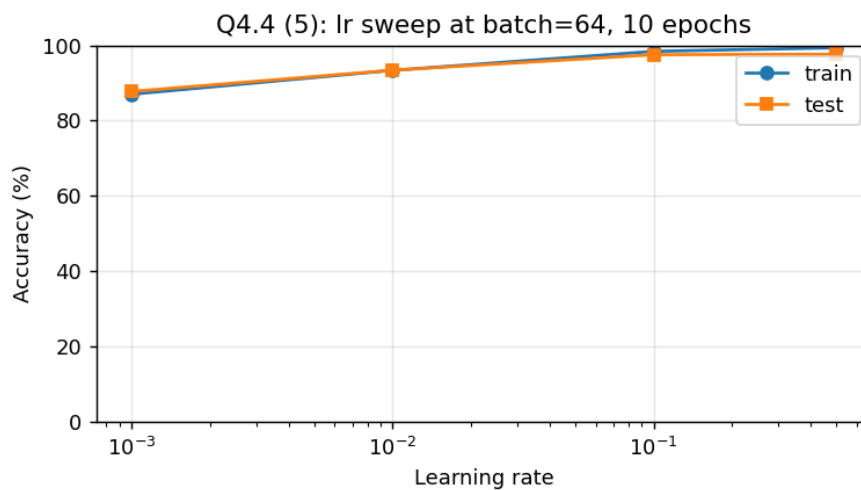


Figure 2: Bumping the learning rate recovers the accuracy lost by batching, exactly as the linear scaling rule predicts.

The same model that capped at 87% with $\alpha = 10^{-3}$ now reaches 97.5% with $\alpha = 10^{-1}$, matching the per-sample baseline from 4.4.1. Each batch is the average of 64 sample gradients, so the per-step magnitude is roughly $1/64$ of the per-sample case, and the learning rate has to be scaled up by a comparable factor to deliver the same effective update. Beyond $\alpha \approx 0.1$ further gains shrink (train climbs, test stays put), which is the first sign of overfitting.

6. Training set reduced to 1000 samples, 20 epochs, batch 64.

α	Train acc (%)	Test acc (%)	Gap (pp)
10^{-3}	29.10	26.09	3.0
10^{-1}	96.10	86.62	9.5

Table 5: 1000 samples, 20 epochs, batch 64, seed 10. Two regimes shown.

With $\alpha = 10^{-3}$ at batch 64 there are only $1000/64 \cdot 20 \approx 312$ weight updates total, far too few to fit even 1000 samples with a 101k-parameter model; both numbers stay at the underfit floor. Switching to the working $\alpha = 10^{-1}$ from 4.4.5 fits the training set almost perfectly (96.1%) while test accuracy

lags ten points behind. This is the textbook overfit regime: ample model capacity, too little data, and no regularisation, so the network memorises noise in the 1000 training examples. The same network on the full 60k-example set generalises fine because more data acts as the regulariser.

Question 4.5 - Regularised MLP (`m1p_adv.py`)

1. What is dropout? Dropout randomly zeroes a fraction p of the activations during each training step, then re-scales the surviving units by $1/(1 - p)$ so the expected magnitude is preserved. At inference time dropout is switched off and every unit participates. The effect is two-fold: it prevents co-adaptation between neurons (no single unit can be relied on, because it might be dropped any iteration), and it acts as an implicit ensemble over the 2^n subnetworks defined by which units are kept.

2. Is deeper / wider always better? No. Adding layers and width increases representational capacity but also increases parameters, memory, training time, and the risk of overfitting and of vanishing or exploding gradients (especially without residual connections or careful initialisation). On MNIST in particular, a 512-512 MLP already saturates the dataset, so a 4096-4096-4096 stack would learn nothing useful that the smaller model misses and would just train slower. Depth is most useful when the data has compositional structure that benefits from learned feature hierarchies, which is more true of images, audio and language than of MNIST digits.

3. AdamW and weight decay. Adam combines momentum (first moment of the gradient) with per-parameter step normalisation (second moment), so each parameter takes a step whose size is adapted to the scale of its own gradient. AdamW is a small fix on top. Vanilla Adam folds an L2 penalty into the gradient before the moment normalisation, which means the effective weight-decay strength ends up tied to the gradient magnitude and breaks the regularisation it was supposed to provide. AdamW decouples the decay and applies it directly to the parameters at update time, so the weight-decay coefficient now behaves as a clean shrinkage term the way the theory expects. The role of the weight-decay parameter is the same as L2 regularisation: it pulls the weights towards zero, which keeps them small and discourages overfitting.

4. Three-seed accuracy on `m1p_adv.py`, 10 epochs.

Seed	Untrained acc (%)	Train acc (%)	Test acc (%)
0	6.65	99.40	97.96
10	8.78	99.38	98.12
42	9.47	99.56	98.24

Table 6: `m1p_adv.py` architecture (Dense 512, Dropout 0.3, Dense 512, Dropout 0.3, Dense 10), AdamW with $\alpha = 10^{-3}$, weight decay 10^{-4} , batch 128, 10 epochs. Roughly 670,000 trainable parameters.

Test accuracy of 97.96-98.24% across seeds sits about a full percentage point above the simple `m1p.py` baseline (97.08-97.25%), and the train-test gap stays around 1.3 pp. The wider 512-512 hidden layers fit the function better than 128 units could, AdamW converges faster than vanilla SGD because the per-parameter step is rescaled by the running variance of the gradient, and dropout regularises the larger network so the extra capacity does not just turn into more overfitting. All three changes pull in the same direction.

Question 4.6 - LeNet-5 (L4_Q6.py)

1. *Parameter count.* The LeNet-5 architecture from the assignment figure has the following parameter breakdown:

Layer	Params	Calculation
C1 (Conv 1→6, 5×5)	156	$5 \cdot 5 \cdot 1 \cdot 6 + 6$
S2 (AvgPool 2×2)	0	no parameters
C3 (Conv 6→16, 5×5)	2,416	$5 \cdot 5 \cdot 6 \cdot 16 + 16$
S4 (AvgPool 2×2)	0	no parameters
F5 (Dense 400→120)	48,120	$400 \cdot 120 + 120$
F6 (Dense 120→84)	10,164	$120 \cdot 84 + 84$
Out (Dense 84→10)	850	$84 \cdot 10 + 10$
Total	61,706	

Table 7: LeNet-5 parameter breakdown. Almost 80% of the parameters live in F5 alone.

The two convolutional stages together hold only $\approx 2.6\text{k}$ parameters even though they do most of the visual feature extraction; the dense layers carry the rest. Compared to the 670k of a 512-512 fully connected MLP this is more than ten times smaller.

2. *Implementation notes.* The implementation follows the architecture above using `flax.linen`. C1 uses SAME padding so the first feature map stays at 28×28 (the original LeNet padded 28×28 to 32×32 ; the modern equivalent is to use SAME on the conv directly). C3 uses VALID padding to get 10×10 , and both pooling layers are 2×2 stride 2. All conv and dense layers use `tanh` activations as the assignment specifies. Training uses AdamW and batched updates of size 128.

3. *Three-seed run at the default hyper-parameters, then a 3×3 LR / weight-decay sweep at seed 10. All runs 10 epochs, batch 128.*

Seed	Untrained acc (%)	Train acc (%)	Test acc (%)
0	6.45	99.32	98.40
10	9.88	99.37	98.27
42	7.89	99.37	98.48

Table 8: LeNet-5, AdamW with $\alpha = 10^{-3}$, $\lambda_{\text{wd}} = 10^{-4}$. Test accuracy spans 0.21 pp across seeds.

α	$\lambda_{\text{wd}} = 0$	$\lambda_{\text{wd}} = 10^{-4}$	$\lambda_{\text{wd}} = 10^{-3}$
10^{-2}	96.49	96.71	97.31
10^{-3}	98.27	98.27	98.25
10^{-4}	96.75	96.75	96.75

Table 9: Test accuracy from the LR $\times$ weight-decay sweep, seed 10, 10 epochs.

Two findings. First, learning rate carries almost all the signal: $\alpha = 10^{-3}$ is the sweet spot at 98.27%, $\alpha = 10^{-2}$ overshoots (96.5-97.3%, the optimiser oscillates near the minimum), and $\alpha = 10^{-4}$ undertrains in 10 epochs (96.75%). Second, weight decay has almost no effect at $\alpha = 10^{-3}$ and is invisible at $\alpha = 10^{-4}$. AdamW applies shrinkage proportional to $\alpha \cdot \lambda_{\text{wd}}$ each step, so at low learning rates the decay scale shrinks proportionally and stops mattering. The only place decay helps materially is at

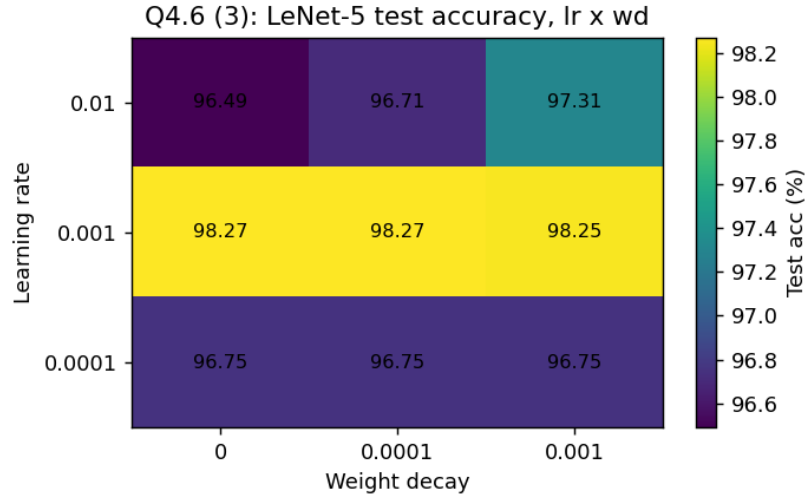


Figure 3: LeNet-5 test accuracy as a function of learning rate and weight decay. The $\alpha = 10^{-3}$ row is uniformly the best regardless of weight decay.

$\alpha = 10^{-2}$, where the stronger regularisation damps the oscillation and recovers about 0.8 pp.

4. LeNet-5 vs MLP comparison.

Model	Params	Best test acc (%)	Configuration
<code>mlp.py</code> (per-sample SGD)	$\approx 101k$	97.25	seed 42, $\alpha = 10^{-3}$, 10 epochs
<code>mlp.py</code> (batched, scaled LR)	$\approx 101k$	97.65	seed 10, $\alpha = 0.5$, batch 64
<code>mlp_adv.py</code> (Dropout + AdamW)	$\approx 670k$	98.24	seed 42, $\alpha = 10^{-3}$, $\lambda_{wd} = 10^{-4}$
LeNet-5	61.7k	98.48	seed 42, same hyper-params as above

Table 10: Best test accuracy reached by each model on this lab. LeNet-5 wins despite having an order of magnitude fewer parameters than the deepest MLP.

Even though LeNet-5 has roughly ten times fewer parameters than the wide MLP (61.7k vs 670k), it generalises a few tenths of a percentage point better. The reason is inductive bias. A convolutional layer encodes two priors: pixels close together are more related than pixels far apart, and the same feature can appear at any spatial position. Both priors are true for digit images, so the parameter budget goes into filters that match the structure of the input instead of being burned on irrelevant input-pixel-to-hidden-unit edges. The MLP has none of this. Every connection is learned independently, and most of those 670k parameters re-discover correlations the convolution captures for free. On MNIST that wastage costs only a few tenths of a percentage point. On harder vision data (CIFAR-10, ImageNet) where the structure is richer, the same inductive bias is the difference between a model that trains and one that does not.

LLM Usage Disclosure

I used Claude Code on this lab for:

- Cross-checking the Flax conv / pool API and the AdamW signature in Optax against the documentation while implementing LeNet-5.

- Editing the report (tightening phrasing, trimming filler).